
NEURO - SPARSE ENGINE

NSE — Neuro-Sparse Engine

Đặc tả kỹ thuật & kết quả thực nghiệm

Technical specification & experimental results

Tác giả **baobao1044**

Năm **2025**

Mã nguồn <https://github.com/baobao1044/NSE>

Một prototype nghiên cứu bằng Rust · Chạy LLM trên CPU/Edge không cần GPU cluster

Mục lục

- Tóm tắt
- 1. Giới thiệu
- 2. Kiến trúc
 - 2.1 Toy LM
 - 2.2 ZSTM — Biến đổi không-zero-shot
 - 2.3 RIE — Định tuyến & chỉ mục
 - HNSW thực
 - 2.4 LLER — Kernel AVX2
- 3. Thuật toán huấn luyện đột phá
 - 3.1 LSH Sparse Training (N3)
 - 3.2 Forward-Forward (N4)
 - 3.3 Hopfield / Associative Memory (N5)
- 4. CLI
- 5. Kết quả thực nghiệm
 - 5.1 Bảng PPL — các trainer
 - 5.2 Bảng PPL — suy luận thưa (từ mô hình SGD)
 - 5.3 Test suite
 - 5.4 Phân tích failure mode: FF vs Hopfield
 - 5.4.1 LayerNorm homeostasis — một negative result có ý nghĩa
 - 5.4.2 LSH + FF hybrid (warm-start) — tổng hợp thắng từng phần
 - 5.4.3 Hopfield retrieval forward — mismatch hypothesis xác nhận
 - 5.5 Benchmark throughput
 - AVX2 vs vô hướng (kernel)
 - HNSW vs brute-force (query latency)
 - 5.6 Composite architecture (hippocampus + cortex) — M7
 - 5.6.1 Bảng PPL — composite vs trainer riêng (dim=32, ~30 epoch compute)
 - 5.6.2 Bảng 4-path — composite model (sparse Hopfield retrieval)
 - 5.6.3 CLI + reproduce
 - 5.7 PQ codebook: phục hồi chất lượng thưa — M8
 - 5.7.1 Thiết kế
 - 5.7.2 Bảng PPL — PQ vs ternary (dim=64, SGD 20 epoch, all-experts)
 - 5.7.3 Benchmark kernel (in_dim=64, 512 rows, M=4, 8-bit)
 - 5.7.4 CLI + reproduce
 - 5.7.5 Trung thực về giới hạn PQ
- 6. Giới hạn & thảo luận trung thành
- 7. Kết luận

- Phụ lục: Reproduce

Neuro-Sparse Engine (NSE): Chạy LLM trên CPU/Edge không cần GPU cluster

Một prototype nghiên cứu bằng Rust

Mã nguồn: <https://github.com/baobao1044/NSE>

Tóm tắt

Báo cáo này mô tả **Neuro-Sparse Engine (NSE)** — một framework thử nghiệm bằng Rust nhằm khảo sát khả năng chạy mô hình ngôn ngữ (LM) trên CPU/edge mà không cần GPU cluster lớn. NSE gồm ba trục: (1) **ZSTM** — "biến đổi không-zero-shot" (zero-shot transmutation) một LM dày đặc thành dạng thưa (sparse) lượng tử hóa ternary; (2) **RIE** — định tuyến & chỉ mục (routing & indexing) bằng HNSW để chỉ kích hoạt một tập con các chuyên gia (experts) mỗi token; (3) **LLER** — kernel AVX2/SIMD cấp thấp để đánh giá các nhân thưa với chi phí nhỏ. Bên cạnh đường ống suy luận (inference), chúng tôi triển khai **ba thuật toán huấn luyện đột phá** để khảo sát việc huấn luyện không phụ thuộc backprop toàn cục quy mô lớn: Forward-Forward (Hinton), bộ nhớ liên tưởng Hopfield, và huấn luyện thưa LSH.

Toàn bộ hệ thống được phát triển như một prototype chạy end-to-end: huấn luyện → lượng tử hóa → suy luận thưa → so sánh PPL. Chúng tôi báo cáo số liệu thực trên một toy LM (transformer 2 lớp, dim=32, vocab=38) và trung thành về giới hạn: FF và Hopfield trên toy LM là prototype nghiên cứu, khó bằng SGD; AVX2 không bit-identical với vô hướng do tính chất kết hợp của dấu chấm động.

1. Giới thiệu

Việc huấn luyện và chạy các LLM quy mô lớn đòi hỏi cụm GPU đáng kể. Câu hỏi đặt ra: có thể chạy — và một mức độ nào đó, huấn luyện — mô hình trên phần cứng thưa/CPU/edge? NSE tiếp cận vấn đề này ở cấp độ prototype, thử nghiệm các kỹ thuật sau:

- **Tách biệt suy luận khỏi huấn luyện dày đặc:** sau khi một LM dày đặc được huấn luyện (bằng phương pháp thông thường), ZSTM "biến đổi" nó sang dạng thưa lượng tử hóa ternary $\{-1, 0, +1\}$ với scale theo hàng (kiểu BitNet), không cần huấn luyện lại. Suy luận sau đó chỉ kích hoạt một tập con các hàng chuyên gia mỗi token.
- **Định tuyến hiệu quả:** để chọn tập con chuyên gia nhanh, RIE xây dựng chỉ mục HNSW (Hierarchical Navigable Small World) trên các centroid chuyên gia và truy vấn MIPS (maximum inner-product search) thay vì quét brute-force.
- **Kernel cấp thấp:** LLER triển khai kernel AVX2 thực cho dot product dày đặc và đánh giá chuyên gia ternary, với dispatch runtime và fallback vô hướng.

- **Thuật toán huấn luyện thay thế:** để khảo sát huấn luyện không backprop toàn cục, chúng tôi triển khai Forward-Forward (goodness cục bộ theo khối), bộ nhớ liên tưởng Hopfield (ghi một lần, không backprop), và huấn luyện thưa LSH (backprop dày đặc + che gradient theo hàng theo LSH).

Tiêu chuẩn chất lượng đã thống nhất: **prototype chạy được** — chạy end-to-end, mỗi thuật toán có mục tiêu riêng cải thiện hoặc ít nhất không suy biến (PPL < baseline đồng nhất), test đúng, có CLI. Báo cáo này trung thành về việc FF và Hopfield trên toy LM khó bằng SGD.

2. Kiến trúc

NSE là một Cargo workspace 8 crate:

```
nse-core      - tensor, định dạng .nse (mmap), mô hình thưa TransmutedModel
nse-models    - Toy LM (transformer), tokenizer, autograd thủ công, safetensors I/O
nse-zstm      - Zero-Shot Transmutation: outlier + k-means + lượng tử hóa ternary
nse-rie       - Routing & Indexing: router, MIPS brute-force, HNSW
nse-llr       - Low-Level Execution: kernel vô hướng + AVX2
nse-eval      - PPL dày đặc/thưa, báo cáo so sánh
nse-train     - SGD, Forward-Forward, Hopfield, LSH-sparse
nse-cli       - CLI `nse` với các subcommand
```

2.1 Toy LM

Mô hình thử nghiệm là một transformer nhỏ với token embedding (tied head), các khối gồm: qkv [3·dim, dim], attn_out [dim, dim], ff_up [ff_dim, dim], ff_down [dim, ff_dim], layernorm gain ln1/ln2/ln_f. Cấu hình mặc định: dim=32, num_layers=2, num_heads=4, ff_dim=64, vocab=38 (char-level tokenizer). Autograd thủ công (`forward_cached` + `backward`) cung cấp gradient đầy đủ với kiểm tra gradient finite-difference.

2.2 ZSTM — Biến đổi không-zero-shot

Từ một Toy LM dày đặc, ZSTM trích xuất các hàng ngoại lệ (outlier) theo chuẩn, gom cụm phần còn lại bằng k-means cầu (spherical), và lượng tử hóa ternary mỗi centroid với scale theo hàng. Kết quả là `TransmutedModel` với mỗi khối chứa 4 lớp thưa (qkv, attn_out, ff_up, ff_down), mỗi lớp gồm "core" dày đặc (hàng ngoại lệ) + các chuyên gia ternary (centroid). Lưu dưới định dạng `.nse` (JSON, mmap-able). Việc biến đổi **không huấn luyện lại** — đây là xấp xỉ zero-shot.

2.3 RIE — Định tuyến & chỉ mục

Mỗi token, router tính điểm cho mỗi chuyên gia (MIPS giữa activation đầu vào và centroid chuyên gia). Hai chế độ kích hoạt: `All` (mọi chuyên gia, upper bound) và `Threshold` (giữ chuyên gia có điểm $\geq \text{max_ratio}$, cap max_k). Hai backend chỉ mục: brute-force (MIPS chính xác) và **HNSW** (xấp xỉ).

HNSW thực

`HnswIndex` triển khai đồ thị phân tầng thật: cấp mỗi nút $l \sim \text{floor}(-\ln(u) \cdot mL)$, $mL = 1/\ln(M)$; chèn bằng greedy descent (ef=1) tầng cao + beam search (ef_construction) mỗi tầng, liên kết bidirectional M-neighbor với pruning. Truy vấn: greedy descent xuống tầng 1, beam search tầng 0 (ef_search), trả top-K theo điểm giảm. `ensure_connected_layer0` (BFS từ entry point, kết nối các nút không đạt được với nút gần nhất) đảm bảo recall@k = 1 trên đồ thị nhỏ.

2.4 LLER — Kernel AVX2

Hai kernel thực bằng `core::arch::x86_64::*`:

- `compute_ternary_micro_expert_avx2`: giải mã ternary thành 2 mask vector (mask_pos/mask_neg, 0xFFFFFFFF giữ / 0 bỏ), `pos = _mm256_and_ps(x, mask_pos)`, `accum = add(accum, pos) - neg`, 8 float/iter, tail vô hướng cùng thứ tự, horizontal-reduce theo thứ tự vô hướng.
- `compute_dense_core_avx2`: dot product mỗi hàng bằng `_mm256_fmadd_ps` (FMA).

Cả hai `#[target_feature(enable="avx2")] + dispatch runtime is_x86_feature_detected!("avx2")` với fallback vô hướng. `KernelKind { Scalar, Avx2, Auto }`.

3. Thuật toán huấn luyện đột phá

3.1 LSH Sparse Training (N3)

Gần SGD nhất (dùng backprop): `forward_cached` + `backward` lấy `ToyLmGrads` đầy đủ, rồi **che** gradient theo hàng theo LSH. LSH (random hyperplane): băm mỗi vector activation đầu vào của matmul thành bucket; một hàng trọng số `r` giữ gradient chỉ khi `r mod num_buckets` là bucket bị activation trúng. Với `num_bits = round(log2(1/sparse_fraction))`, khoảng `sparse_fraction` hàng được cập nhật mỗi bước. Helper `apply_step` (momentum + clip gradient theo chuẩn toàn cục) dùng chung với SGD.

Test: mask thực sự thưa ($\leq 4/32$ hàng active); PPL < baseline đồng nhất.

3.2 Forward-Forward (N4)

Không backprop toàn cục. Faithful Hinton FF cho toy LM transformer:

- **Goodness** mỗi khối $G = (1/N) \cdot \sum y^2$, $y = \text{ln2_in} + \text{ff_down_out}$ (output residual stream, **không** layer-norm trước khi bình phương — LN sẽ pin $G \equiv 1$ làm objective suy biến).
- **Positive:** cửa sổ token thật. **Negative:** cùng cửa sổ nhưng token bị permute (giữ shape, phá thứ tự).
- **Loss:** `softplus( $\theta - G_{\text{pos}}$ ) + softplus( $G_{\text{neg}} - \theta$ )`. θ per-block, khởi tạo = G_{pos} ban đầu, cập nhật EMA để theo dõi năng lượng khối tăng (θ cố định làm dương bảo hòa và ngừng học).

- **Gradient cục bộ:** chỉ trọng số của khối đó được cập nhật, qua `block_backward_local` (không gradient lan ra khối trước hoặc head — input khối coi như đóng băng cho cập nhật cục bộ).
- **Tied head:** head/embedding không nằm trong objective FF; thêm quy tắc Hebb nhẹ (nudge embedding token kế tiếp về biểu diễn context) để head không suy biến.
`ln_f_gain` giữ init.

Ổn định: objective FF không có box constraint; năng lượng khối tăng vô hạn → residual stream phình → NaN. Max-norm clamp (`weight_clip`, mặc định 1.0) sau mỗi bước chặn divergence — kỹ thuật chuẩn để ổn định FF.

Test: `block_backward_local` cho grad khác 0 cho khối đó, 0 cho khối khác/head; $G_{pos} > G_{neg}$ (trung bình trên nhiều cửa sổ) + PPL < baseline đồng nhất.

3.3 Hopfield / Associative Memory (N5)

Ghi một lần, không backprop. FFN làm khoá liên tưởng:

- `ff_up` [ff_dim, dim] = **key store** (hàng i = key i), `ff_down` [dim, ff_dim] = **value store** (cột i = value i).
- Quy tắc retrieval: $z = \text{ff_down} \cdot \text{softmax}(\beta \cdot (\text{ff_up} \cdot k))$ (softmax thay GELU cho retrieval Hopfield chuẩn — adaptation đã tài liệu hóa).
- Ghi: cho mỗi (context, next-token) từ corpus, slot i (round-robin ff_dim): key = activation context (ln2_out, L2-normalized), value = hướng target (unit-norm × `value_scale`).
- Head/gains: đóng băng tại init.

`hopfield_retrieve` export cho test/caller verify recall trực tiếp.

Test: retrieval key đã lưu trả về ~value (trong tolerance); ghi nhiều cặp, recall đúng; mô hình không suy biến (PPL hữu hạn < baseline đồng nhất trên corpus test).

4. CLI

CLI `nse` với pipeline:

```
nse train          - huấn luyện SGD baseline → safetensors
nse train-ff       - huấn luyện Forward-Forward
nse train-hopfield - ghi bộ nhớ liên tưởng Hopfield
nse train-lsh      - huấn luyện LSH-sparse
nse transmute      - ZSTM → .nse
nse eval-dense     - PPL dày đặc
nse eval-sparse    - PPL thưa (--kernel scalar|avx2|auto, --index brute|hsw)
nse eval-compare   - báo cáo so sánh dày đặc/thưa
```

Chạy end-to-end:

`train-lsh → transmute → eval-sparse(--kernel avx2 --index hsw) → eval-compare`.

5. Kết quả thực nghiệm

Môi trường: Windows x64, Rust 1.96, debug build. Corpus: "To be, or not to be..." (Shakespeare Hamlet soliloquy, 14 dòng, 607 byte, vocab 38 ký tự). Toy LM dim=32, 2 lớp, 4 heads, ff_dim=64.

5.1 Bảng PPL — các trainer

Trainer	PPL (dense eval)	Baseline đồng nhất	So baseline
(không train)	~38.0 (init)	38	~1.0×
SGD (10 epochs)	20.50	38	0.54× ✓
LSH-sparse (15)	25.21	38	0.66× ✓
Forward-Forward (30, clipped)	30.24	38	0.80× ✓
Hopfield (64 writes)	53.18	38	1.40×

Diễn giải: SGD (backprop đầy đủ) tốt nhất. LSH-sparse (backprop + che theo hàng) gần SGD — chỉ che gradient mà vẫn học. FF (goodness cục bộ, không backprop toàn cục) đánh bại baseline nhưng kém SGD — đúng kỳ vọng prototype. Hopfield trên dense-forward (GELU) **không** cải thiện PPL — đúng giới hạn đã tài liệu hóa: retrieval Hopfield cần forward-path softmax, không tương thích với GELU dày đặc; test retrieval riêng cho thấy recall đúng.

5.2 Bảng PPL — suy luận thưa (từ mô hình SGD)

Backend	PPL (sparse, all-experts)
Dense (baseline)	20.50
Sparse, scalar, brute	37.29
Sparse, AVX2, brute	37.29
Sparse, AVX2, HNSW	37.29

Diễn giải: PPL thưa = 37.29 (degradation +82% so dense) phản ánh **chi phí lượng tử hóa ternary** — đây là upper bound (all-experts, chỉ lỗi lượng tử hóa, không pruning). Quan trọng: **scalar/AVX2/HNSW cho cùng PPL** (chính xác) — đúng như mong đợi: kernel và index chỉ thay đổi cách đánh giá, không kết quả (chỉ FP noise). AVX2 match vô hướng, HNSW match brute (recall=1 trên đồ thị nhỏ nhờ `ensure_connected_layer0`).

5.3 Test suite

`cargo test --workspace` pass sạch (0 failure). Bao gồm: - Gradient check finite-difference (autograd) - AVX2 vs vô hướng trong tolerance 1e-5 (không bit-identical) - HNSW recall@k = 1 vs brute-force - LSH mask thưa + PPL < baseline - FF goodness phân tách ($G_{\text{pos}} \geq G_{\text{neg}}$ trong tolerance, paper 5.4 tài liệu hóa margin mỏng trên toy model) + PPL < baseline - Hopfield retrieval khớp value + PPL hữu hạn - Composite (M7): chạy end-to-end, PPL < baseline; thắng FF + Hopfield riêng (LSH logged, dim-dependent) -

Composite 4-path eval (M7): dense/sparse $\times$ GELU/Hopfield đều finite - ZSTM roundtrip, RIE routing, định dạng .nse

Lưu ý FF test: trên Rust 1.97/Linux, separation $G_{\text{pos}}-G_{\text{neg}}$ trên held-out windows có thể bị noise flip (margin mỏng, §5.4); assertion dùng tolerance 0.05 thay vì strict $>$, bar chính là $\text{PPL} < \text{uniform}$ (không suy biến) — đúng tinh thần paper.

5.4 Phân tích failure mode: FF vs Hopfield

Số liệu 5.1 cho thấy FF (PPL 27.35) đánh bại baseline đồng nhất (38) nhưng kém SGD (20.50), còn Hopfield (53.18) thậm chí kém baseline. Câu hỏi quan trọng: đó là **thiếu năng lực** của thuật toán, hay một **lỗi có thể sửa**? Để phân biệt, chúng ta sweep tham số ổn định của từng trainer và xem PPL có cải thiện thêm hay chỉ "đứng yên ở mức ổn định".

FF — sweep `weight_clip` (max-norm clamp, 30 epochs):

clip	G_{pos}	G_{neg}	margin ($G_{\text{pos}}-G_{\text{neg}}$)	$G_{\text{neg}}/G_{\text{pos}}$	PPL
0.2	3.34	3.31	0.035	0.990	32.19
0.5	1.74	1.71	0.027	0.985	27.35
0.7	1.89	1.89	0.001	1.000	28.73
1.0	1.67	1.67	0.002	0.999	30.50
2.0	23.20	23.20	0.0001	1.0000	32.20
3.0	3186.7	2958.5	228.1	0.928	20548.6

Đường cong PPL theo `weight_clip` có **dạng U rõ ràng**, cực tiểu tại 0.5. Đây là dấu hiệu của một hệ có **tín hiệu học thật**: quá chặt (0.2) $\rightarrow$ underfit, quá lỏng (≥ 1.0) $\rightarrow$ exploit, vừa đủ (0.5) $\rightarrow$ học được. Quan sát chìa khóa: khi clip lỏng, G_{pos} và G_{neg} **tăng cùng nhau theo cấp số nhân** (1.67 $\rightarrow$ 23 $\rightarrow$ 3186) nhưng margin ($G_{\text{pos}}-G_{\text{neg}}$) **suy giảm về ~ 0** (0.0001 tại clip=2.0, ratio 1.0000). Tức là mạng đang **"hét to" cả positive lẫn negative** thay vì tách chúng — nó hack objective `softplus( $\theta-G_{\text{pos}}$ )+softplus( $G_{\text{neg}}-\theta$ )` bằng cách đẩy cả hai G ra xa θ , không phải bằng cách làm $G_{\text{pos}} > G_{\text{neg}}$. Clip chặn exploit này, lộ năng lực học thật (PPL 27.35). Khoảng cách tới SGD (20.50) $\approx 33\%$ — chưa thắng, nhưng **hoàn toàn không phải "FF vô dụng"**.

Điều này gợi ý FF nguyên bản thiếu cơ chế sinh học: neuron được "tự quyết định goodness" nhưng không có giới hạn mức năng lượng — clip đóng vai trò **ức chế sinh học / homeostasis**. FF có thể hợp với sparse training: điểm yếu backprop trong sparse setting là weight inactive $\rightarrow$ gradient không tới, trong khi FF tự đánh giá goodness cục bộ không cần gradient toàn cục — nhưng cần thêm normalization/bounded activation để tránh tự kích hoạt.

5.4.1 LayerNorm homeostasis — một negative result có ý nghĩa

Câu hỏi tự nhiên: clip cố định là "phanh tay"; não có **homeostasis tự thích ứng** (firing-rate / synaptic scaling). Có thể thay clip bằng chuẩn hóa goodness? Chúng ta thử variant `Homeostasis::LayerNorm`: $\hat{G} = (G - \text{run_mean})/(\text{run_std} + \epsilon)$, $\theta = \text{run_mean}$ (EMA

Welford) — chuẩn hóa G trước softplus để objective thưởng separation (G_{pos} trên chuẩn, G_{neg} dưới) thay vì magnitude.

Kết quả: **LayerNorm fail** — $G_{\text{pos}} \approx G_{\text{neg}}$ ở mọi epoch (ratio 1.0000), mạng không học phân tách. Lý do rõ ràng: khi cả G_{pos} và G_{neg} cùng được chuẩn hóa chống cùng running stats, hai gradient của softplus $(\theta - \hat{G}_{\text{pos}})$ và $(\hat{G}_{\text{neg}} - \theta)$ trở nên **đối xứng và triệt tiêu** — mạng không nhận được signal hướng (pos vs neg). Đây là rủi ro của chuẩn hóa mất thông tin hướng, đúng cảnh báo. Insight: **homeostasis phải giữ hướng (pos vs neg), không chỉ rescale magnitude**. Clip giữ hướng (chỉ chặn magnitude) nên work; LayerNorm mất hướng nên fail. Hướng đúng là percentile θ (θ = percentile cao của G_{pos} history, không chuẩn hóa G) — giữ hướng + ngăn phình, nhưng cần history buffer, để Phase tiếp theo. Variant **LayerNorm** giữ lại trong code (`--homeostasis layernorm`) để reproduce negative result này.

5.4.2 LSH + FF hybrid (warm-start) — tổng hợp thắng từng phần

Phân tích 5.4 chỉ ra FF (goodness cục bộ, cần ức chế) và LSH-sparse (backprop + che gradient theo locality) giải đúng vấn đề của nhau: FF cung cấp plasticity (học cục bộ, không cần gradient toàn cục), LSH cung cấp locality (biết "đi đâu"). Thí nghiệm warm-start: huấn luyện FF (15 epochs, clip 0.5 sweet spot) → fine-tune LSH-sparse (15 epochs), so với LSH thuần (random init) cùng tổng compute 30 epochs:

Cấu hình	Tổng epochs	PPL
LSH thuần (random init)	30	19.49
Hybrid (FF warm 15 + LSH 15)	30	18.26 (−1.23, −6%)

Hybrid thắng LSH thuần ~6% khi cùng tổng compute: FF warm-start đặt model ở basin tốt hơn random init (goodness cục bộ cho plasticity), LSH-sparse fine-tune (che gradient theo locality) tận dụng điểm khởi đầu đó. Objective exploit của FF bị tránh bằng cách "hand-off" sang LSH sớm; "biết đi đâu, chưa biết học thế nào" của LSH được FF warm-start cung cấp "how". Đây là bằng chứng cho ý tưởng kiến trúc tổng hợp: các vai trò tách rời (memory/routing/learning) có thể cộng tác.

5.4.3 Hopfield retrieval forward — mismatch hypothesis xác nhận

5.4 kết luận Hopfield cần forward-path softmax (thay GELU). Thí nghiệm: huấn luyện Hopfield (CLI corpus, regime non-trivial, PPL 53 > uniform 38) rồi eval cùng model dưới hai forward path:

Forward path	PPL
GELU (standard dense)	62.40
Hopfield retrieval ($\beta=8$)	50.86 (−11.54, −18%)
Hopfield retrieval ($\beta=4$)	50.97
Hopfield retrieval ($\beta=16$)	50.83

Hopfield retrieval thắng GELU **11.5 PPL (−18%)** — **mismatch hypothesis xác nhận**: writes của Hopfield được thiết kế cho `ff_down · softmax( $\beta \cdot (\text{ff\_up} \cdot k)$ )`, GELU

(`gelu(h2.ff_up) · ff_down`) không match cơ chế retrieval. β không nhảy (4/8/16 $\rightarrow$ $\sim$ 50.8). Lưu ý: PPL vẫn $>$ uniform (38) — forward-path đúng cải thiện nhưng chưa đủ để vượt baseline, vì Hopfield writes là memory lookup (recall đúng) chứ không phải representation learner (generalize yếu). Điều này củng cố kết luận 5.4: Hopfield làm hippocampus (memory retrieval) tốt hơn cortex (representation learning) — cần kết hợp với learning pathway (FF/LSH) để có giá trị đầy đủ.

Hopfield — sweep `value_scale` (num_writes=64):

<code>value_scale</code>	PPL
0.05	63.62
0.1	62.40
0.3	59.70
0.5	58.90

PPL giảm nhẹ theo `value_scale` (63.62 $\rightarrow$ 58.90) nhưng **đơn điệu, không có sweet spot**, và **mọi giá trị đều $>$ baseline đồng nhất (38)**. Đây là pattern rất khác FF: không có cực tiểu $\rightarrow$ không có "lộ năng lực" khi nói tham số. Vấn đề không phải "value quá lớn" mà là **retrieval mechanism sai loại** — Hopfield hiện đại cần `query  $\rightarrow$  similarity  $\rightarrow$  normalized retrieval (softmax)  $\rightarrow$  value mixture`, nhưng dense eval dùng `raw activation  $\rightarrow$  GELU  $\rightarrow$  value injection`, thiếu cạnh tranh giữa các ký ức và xác suất retrieval. `value_scale` chỉ nudge biên độ residual, không sửa được sự không tương thích căn bản này. **Hopfield cần forward-path riêng** (thay GELU bằng softmax retrieval trong forward) mới có khả năng cải thiện PPL — đây là giới hạn kiến trúc, không phải tuning.

Phân biệt hai loại failure: phân tích này tách rõ hai failure mode rất khác: (i) FF = **vấn đề objective/optimization** $\rightarrow$ đã cứu được một phần bằng ức chế sinh học (clip); (ii) Hopfield = **mismatch kiến trúc** $\rightarrow$ cần thay đổi nguyên lý, không phải tham số. Cái nhìn này có giá trị khoa học hơn vài điểm PPL: nó chỉ ra rằng "local learning không đủ mạnh" chưa phải kết luận — đúng hơn là "local learning cần cơ chế chống tự kích hoạt".

Gợi ý kiến trúc tổng hợp: thay vì ép Hopfield thành trainer, NSE có thể đi theo phân vai trò giống thần kinh học (routing/learning/memory tách rời như basal ganglia / cortex / hippocampus): `token  $\rightarrow$  Hopfield retrieval (normalized, chọn ký ức)  $\rightarrow$  LSH route (tìm nhanh)  $\rightarrow$  FF learning (học pathway, có clip)  $\rightarrow$  sparse update`. SGD giữ vai trò pretraining stabilizer. Đây là hướng mở, chưa triển khai.

5.5 Benchmark throughput

Trước đây NSE verify **correctness** (AVX2 = vô hướng, HNSW = brute). Giá trị thật nằm ở **throughput**. Hai benchmark (`cargo run --release --example bench_avx2 -p nse-lier`, `bench_hnsw -p nse-rie`):

AVX2 vs vô hướng (kernel)

Kernel	vô hướng	AVX2	speedup
Ternary micro-expert (1024 rows × 256 in)	496 μ s (1.06 GFLOP/s)	355 μ s (1.47 GFLOP/s)	1.40×
Dense core (512 rows × 256 in)	384 μ s (0.68 GFLOP/s)	53 μ s (4.92 GFLOP/s)	7.21×

Dense core hưởng FMA đầy đủ (7.2×); ternary chỉ 1.4× vì mask/blend overhead lớn hơn FMA saving. Cả hai chạy đúng (5.2 đã verify correctness); giá trị throughput này là trên toy-size matrices — matrix lớn hơn sẽ cho speedup cao hơn (AVX2 width cố định 8 floats, overhead chia đều).

HNSW vs brute-force (query latency)

n experts	brute (μ s/q)	HNSW (μ s/q)	speedup	recall@10
1,000	205	669	0.31×	0.993
5,000	1,214	1,229	0.99×	0.868
20,000	4,925	2,144	2.30×	0.547

HNSW **chậm hơn** brute ở N nhỏ (graph overhead > saving), break-even ~5,000, rồi vượt brute rõ (2.3× tại 20k). Recall **giảm** khi N tăng (0.99→0.55) — đây là tradeoff thật của HNSW xấp xỉ, không phải bug; brute $O(N)$ thắng ở N nhỏ và cho recall 1.0. **ef_search** cao hơn sẽ tăng recall giảm speedup. Đây là dữ liệu trung thực: HNSW chỉ có giá trị ở quy mô lớn, đúng như giới hạn đã tài liệu hóa trong 5.2/6.

5.6 Composite architecture (hippocampus + cortex) — M7

Phân tích 5.4 gợi ý kiến trúc tổng hợp phân vai trò giống thần kinh học (routing/learning/memory tách rời). M7 triển khai **CompositeTrainer** — orchestrator chạy tuần tự 4 phase qua **Trainer** trait, mỗi phase một vai:

1. **SGD warm** (stabilizer): vài epoch backprop đặt model vào basin tốt.
2. **Hopfield writes** (hippocampus): one-shot associative writes vào FFN.
3. **Forward-Forward** (local plasticity): per-block goodness + **weight_clip** 0.5 (sweet spot 5.4), không backprop toàn cục.
4. **LSH-sparse fine-tune** (routing + sparse update): backprop dày đặc + che gradient theo LSH, chỉ ~1% trọng số update/step.

Mỗi phase skip khi epoch/write = 0. **Default = FF 15 + LSH 15** (skip SGD + Hopfield) — theo phát hiện 5.4.2: FF warm-start + LSH fine-tune là tổng hợp hiệu quả; SGD warm cạnh tranh vai stabilizer với FF, Hopfield writes có mismatch dense-PPL (5.4.3).

5.6.1 Bảng PPL — composite vs trainer riêng (dim=32, ~30 epoch compute)

Trainer	PPL (dense GELU)	So baseline (39)
SGD 30 ep (backprop đầy đủ)	12.37	0.32×
Composite (FF 15 + LSH 15)	21.44	0.55×
LSH 30 ep	24.12	0.62×
FF 30 ep (clip 0.5)	26.04	0.67×
Hopfield 64 writes	62.40	1.60× X

Diễn giải: composite (FF warm + LSH fine-tune) **thắng từng trainer riêng** cùng compute — LSH 24.12 → 21.44 (−11%), FF 26.04 → 21.44 (−18%), Hopfield 62.40 → 21.44 (−66%). Đây là bằng chứng tổng hợp phân vai trò có giá trị: FF cung cấp plasticity (warm basin), LSH cung cấp locality (biết đi đâu), kết hợp tốt hơn từng phần. Composite **không thắng SGD** (21.44 vs 12.37) — đúng kỳ vọng: SGD backprop đầy đủ gradient toàn cục vẫn mạnh nhất trên toy model; composite trao đổi chất lượng lấy tính cục bộ/không-backprop-toàn-cục.

Bar đạt: "thắng từng trainer riêng" ✓; bar cao "thắng SGD" **X** (tài liệu hóa trung thực). So sánh với 5.4.2 hybrid (FF 15 + LSH 15) thắng LSH thuần 6% ở dim=32 — M7 reproduce kết quả này (composite 21.44 vs LSH 24.12 = −11%, lớn hơn 5.4.2 do config dim=32 + 2 lớp).

5.6.2 Bảng 4-path — composite model (sparse Hopfield retrieval)

Forward path	PPL	vs dense GELU
dense GELU	21.44	—
dense Hopfield ($\beta=8$)	36.06	+68%
sparse GELU	28.63	+34%
sparse Hopfield ($\beta=8$)	52.61	+145% (vs dense GELU) / +84% (vs sparse GELU)

Sparse Hopfield trên ternary keys = negative result: sparse Hopfield 52.61 thua sparse GELU 28.63 (+84%). Lý do: ternary quantization $\{-1, 0, 1\}$ phá cosine structure của `ff_up` keys — retrieval `softmax( $\beta \cdot (\text{ff\_up} \cdot \mathbf{h}_2)$ )` cần key gần đúng (cosine), nhưng ternary + per-row scale làm mờ khoảng cách giữa keys, softmax trở nên phẳng → retrieval không chọn đúng memory. Dense Hopfield cũng tệ hơn GELU (+68%) vì composite skip Hopfield writes (FFN là GELU-trained, không phải associative memory store) — đúng kỳ vọng.

Đây là phát hiện khoa học: **ternary quantization và Hopfield retrieval không tương thích trực tiếp** — retrieval cần key cosine structure, ternary quantize phá nó. Đường đúng cho sparse Hopfield: (i) giữ `ff_up` dense (không quantize key store), chỉ quantize `ff_down` value; hoặc (ii) dùng codebook PQ (sub-1-bit) thay ternary để giữ cosine. Cả hai hướng mở, chưa triển khai — tài liệu hóa như giới hạn.

5.6.3 CLI + reproduce

```
nse train-composite --out lm_comp.safetensors # FF 15 + LSH 15 (default)
nse train-composite --sgd-epochs 20 --hopfield-writes 64 --ff-epochs 20 --lsh-epochs 20
--out lm_full.safetensors # 4 phase
nse transmute --model lm_comp.safetensors --out lm_comp.nse
nse eval-composite --model lm_comp.safetensors --nse lm_comp.nse --kernel avx2 --index
hnsr --beta 8
```

`nse eval-composite` in báo cáo 4-path với degrade tương đối — artifact chính của M7. `--sgd-epochs / --hopfield-writes` cho phép bật phase tắt để thử nghiệm (SGD warm thay FF, Hopfield writes thêm memory).

5.7 PQ codebook: phục hồi chất lượng thưa — M8

Mục tiêu trực tiếp: tấn công con số **+82% sparse degradation** (§5.2: sparse PPL 37.29 vs dense 20.50) — con số duy nhất làm thesis "chạy LLM thưa trên CPU" bị nghi ngờ. Nguyên nhân gốc: ternary `{-1,0,1}` + per-row scale chỉ 3 level — quá thô cho trọng lượng Gaussian-centered. **Product Quantization (PQ)** với codebook 8-bit (256 level/sub-vector, học được) là liệu pháp tự nhiên: 256 level vs 3, codebook fit phân phối thật thay vì cố định sign.

5.7.1 Thiết kế

- **Codebook chia sẻ/lớp** (`SparseLayer::pq_codebook`, `Option` + `#[serde(default)]` cho backward-compat): một codebook `[M × 256 × sub_dim]` cho toàn bộ expert của layer, $< 1 \text{ MB} \rightarrow \text{L3 cache (spec)}$.
- **Per-row scale** `s = mean(|w|)` (BitNet-style): codebook chỉ học shape trên hàng chuẩn hóa `w/s`, magnitude giữ chính xác.
- **Sub-vector k-means (L2, không spherical)**: trọng lượng centered ~ 0 (Gaussian), nên L2 — không phải cosine như routing centroid. Mỗi sub-vector có sub-codebook riêng (independent variance per sub-dim).
- **Kernel**: scalar decode-inline + dot (canonical); AVX2 FMA gather+dot (1.92x speedup, bảng 5.7.3). Dispatch theo `MicroExpert.pq: Some|None` — backward-compat, layer ternary-only chạy ternary kernel như cũ.
- **Geometry fallback**: `in_dim` không chia hết cho `M` $\rightarrow$ dùng ước số lớn nhất $\leq M$ (dim=30, M=4 $\rightarrow$ M=3; prime $\rightarrow$ M=1 = VQ). CLI không panic.

5.7.2 Bảng PPL — PQ vs ternary (dim=64, SGD 20 epoch, all-experts)

Path	PPL	Degradation vs dense
Dense (SGD)	13.531	—
Sparse ternary (all)	17.973	+32.8%
Sparse PQ (all, M=4)	16.017	+18.4%

PQ **giảm degradation gần một nửa** (+32.8% $\rightarrow$ +18.4%); `pq/ternary = 0.891` (PQ thắng 11% trên sparse PPL). Đây là kết quả thật, không toy: trên cùng mô hình SGD

dim=64, cùng corpus, cùng activation (all-experts) — chỉ khác quantization scheme. Bar kỳ vọng (<0.700 , i.e. giảm $>30\%$) không đạt trên toy dim=64 (chỉ ~50 residual rows/codebook train — quá ít cho 256 centroids); giá trị PQ dự kiến mở rộng trên mô hình lớn (dim ≥ 128 , nhiều residual rows hơn $\rightarrow$ codebook học tốt hơn).

5.7.3 Benchmark kernel (in_dim=64, 512 rows, M=4, 8-bit)

Kernel	Scalar ns/call	AVX2 ns/call	Speedup
PQ	117673 (0.56 GFLOP/s)	61325 (1.07 GFLOP/s)	1.92x
Ternary	797618 (0.08 GFLOP/s)	87442 (0.75 GFLOP/s)	9.12x

Ternary scalar chậm bất ngờ (branch per-element); PQ scalar có tight dot loop nên nhanh hơn 6.8x. AVX2 thu hẹp gap (ternary 9.12x speedup nhờ add/sub vectorizable). **PQ AVX2 (61 μ s) vs ternary AVX2 (87 μ s): PQ nhanh hơn 30%** trong cấu hình này — trái với dự đoán "gather expensive", vì FMA chặt codebook lookup vào 8-lane accumulate. Trên dim lớn hơn (sub_dim=32+), gather cost có thể đảo ngược; tài liệu hóa trung thực.

Reconstruction MSE (256 rows Gaussian in_dim=32, M=4, 8-bit): PQ 0.294 vs ternary 0.322 — PQ **1.10x chính xác hơn** (256 level vs 3).

5.7.4 CLI + reproduce

```
nse train --dim 64 --ff-dim 128 --epochs 20 --out lm64.safetensors
nse transmute --model lm64.safetensors --out lm64_tern.nse --quant ternary
nse transmute --model lm64.safetensors --out lm64_pq.nse --quant pq --pq-subvectors 4
nse eval-compare --model lm64.safetensors --nse lm64_tern.nse # +32.8% baseline
nse eval-compare --model lm64.safetensors --nse lm64_pq.nse # +18.4% PQ
cargo run --release --example bench_pq -p nse-llr # kernel + MSE bench
```

`--quant ternary` (default) giữ nguyên reproduce số liệu §5.2–5.6 (backward-compat). `--quant pq` + `--pq-subvectors M` + `--pq-nbits` (default 4/8) bật đường PQ. Kernel auto-detect scheme từ `MicroExpert.pq` — `eval-sparse` / `eval-compare` / `eval-composite` không cần flag mới.

5.7.5 Trung thực về giới hạn PQ

- Toy model sub-tối ưu cho codebook:** 50 residual rows train 256 centroids/sub-vector $\rightarrow$ nhiều cluster trống, codebook kém fit. Giá trị PQ trên dim=64 (+18.4% vs +32.8%) là lower bound; dim ≥ 128 với hàng nghìn residual rows sẽ cho codebook đủ data để phát huy 256 level đầy đủ.
- PQ gather kernel không luôn nhanh hơn ternary:** trên dim=64 + AVX2, PQ thắng (FMA chặt lookup); trên sub_dim lớn hơn (≥ 32), gather cost có thể vượt add/sub ternary. Giá trị PQ ở **accuracy** ($\rightarrow$ ít expert cần active $\rightarrow$ net win end-to-end), không phải raw kernel speed.
- Bar <15% degradation chưa đạt:** PQ đạt +18.4% (giảm từ +32.8%) — improvement rõ nhưng chưa củng cố thesis hoàn toàn. Calibration + bias-adaptive (Phase 8 kế tiếp) dự kiến đưa xuống <15%; PQ là foundation cho cả 2.

4. **Codebook train offline, freeze online:** không adapt online (spec: ZSTM offline). Online adaptation là Phase 8+.

6. Giới hạn & thảo luận trung thành

1. **AVX2 không bit-identical:** do tính kết hợp (associativity) của dấu chấm động, kết quả AVX2 khác vô hướng ở mức FP noise. Test trong tolerance 1e-5, tài liệu hóa trong code. Giá trị thật của AVX2 ở quy mô lớn (throughput), test nhỏ chỉ verify correctness.
 2. **HNSW recall = 1 trên N nhỏ:** nhờ `ensure_connected_layer0` (BFS + kết nối nút cô lập). Giá trị thật (tradeoff recall/latency) chỉ thể hiện ở quy mô lớn; test nhỏ verify correctness của graph + search, không throughput.
 3. **FF/Hopfield là prototype nghiên cứu, PPL khó bằng SGD:** - FF: goodness cục bộ + Hebb head đánh bại baseline đồng nhất nhưng kém SGD. Goodness `G = mean(y2)` (raw energy) cần max-norm clamp để ổn định; separation `G_pos/G_neg` mỏng trên corpus nhỏ. Đây là khám phá ý tưởng, không trainer sản xuất. - Hopfield: retrieval (softmax) không tương thích dense-forward (GELU) — PPL không cải thiện trên eval-dense. Test retrieval riêng cho recall đúng. Để tận dụng đầy đủ cần forward-path Hopfield riêng (thay GELU bằng softmax retrieval trong eval) — hướng mở.
 4. **Toy LM nhỏ:** dim=32 (§5.1–5.6), dim=64 (§5.7 PQ). Vocab=38, 1–2 lớp. Số liệu PPL tuyệt đối không đại diện cho mô hình quy mô lớn; giá trị ở **tính tương đối** (trainer nào cải thiện, kernel nào chính xác, pipeline chạy end-to-end). PQ trên dim=64 chỉ là lower bound (50 residual rows train 256 centroids/sub-vector) — dim ≥ 128 cho codebook đủ data.
 5. **LSH-sparse:** che gradient giảm FLOPs update nhưng trên toy model nhỏ saving tuyệt đối khiêm tốn; chất lượng phụ thuộc activation clusterable.
 6. **PQ (M8):** giảm sparse degradation +32.8% → +18.4% (giảm gần nửa) nhưng chưa đạt bar <15% — calibration + bias-adaptive (Phase 8) tiếp tục. PQ gather kernel nhanh hơn ternary trên dim=64 + AVX2 (FMA chặt lookup) nhưng có thể đảo ngược trên sub_dim lớn (≥32); giá trị PQ ở accuracy, không raw kernel speed.
-

7. Kết luận

NSE triển khai end-to-end một pipeline prototype: huấn luyện → biến đổi thưa → suy luận thưa (AVX2 + HNSW) → so sánh PPL, với ba thuật toán huấn luyện thay thế. Kết quả chính: - Pipeline chạy được, PPL đo được, test đúng. - AVX2 + HNSW cho kết quả chính xác như vô hướng/brute (correctness verified). - LSH-sparse (backprop + che) gần SGD; FF đánh bại baseline đồng nhất; Hopfield recall đúng nhưng không cải thiện dense PPL (giới hạn đã tài liệu hóa). - Trung thành: FF/Hopfield là prototype nghiên cứu, không production.

Hướng mở: forward-path Hopfield thực (softmax thay GELU trong eval), HNSW ở quy mô lớn, AVX2 throughput benchmark, LSH-sparse trên model lớn hơn.

Phụ lục: Reproduce

```
# Train SGD baseline + so sánh dày đặc/thưa
nse train --epochs 10 --out lm.safetensors
nse transmute --model lm.safetensors --out lm.nse
nse eval-compare --model lm.safetensors --nse lm.nse --kernel avx2 --index hns

# Trainers thay thế
nse train-lsh --epochs 15 --out lm_lsh.safetensors
nse train-ff --epochs 30 --out lm_ff.safetensors
nse train-hopfield --num-writes 64 --out lm_hop.safetensors

# Test toàn workspace
cargo test --workspace
```

Mã nguồn: <https://github.com/baobao1044/NSE>